RESEARCH MEETING

Hetis: Fine-grained and Dynamic Parallelism

LLM Serving in Heterogeneous GPU Clusters

Yu-Jui Liang
Advisor: Hsiang-Yun Cheng
National Taiwan University | 21 Oct 2026

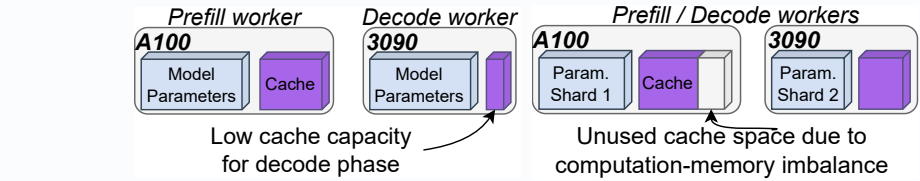
Presentation Roadmap

HETIS / OVERVIEW

1. **Background / Motivation**
   KV cache imbalance and module-specific performance gaps.
2. **Methodology**
   Primary parallelism, head allocation and KV cache management.
3. **Experimental Results**
   Latency, usable cache capacity, sensitivity and overhead.
4. **Discussion / Conclusion**
   Main findings, comparison with ThunderServe and research directions.

Source: arXiv:2509.08309 (Hetis); ThunderServe in the closing comparison.

Coarse Splitting Wastes KV Cache Capacity



(a) Phase-splitting **(b) Parameter-splitting**

Fig. 1; Section 2.3: Two sources of memory inefficiency

- Phase splitting stores separate model copies and confines decode KV cache to decode workers.
- In the FP16 7B example, a 24 GB 3090 has only about 10 GB left for KV cache.
- Parameter splitting balances compute, but fixes where each request stores its KV cache.
- One GPU can run out of cache while another still has free memory.

Balancing computation alone does not guarantee usable KV cache capacity.

Measured GPU Gaps and a Partition Example

Table 1. OPT-2.7B, one iteration through all layers. Prefill batch 3; decode batch 25.

Device	Memory	Time (Prefill)	Time (Decode)
A100	80 GB	0.06 s	0.0097 s
3090	24 GB	0.147 s	0.0143 s
P100	12 GB	1.47 s	0.077 s

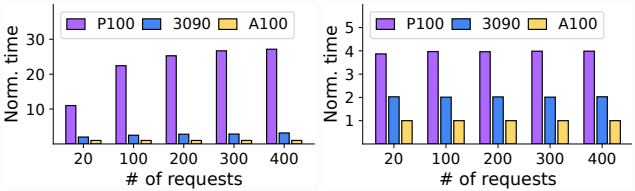
A100 vs. 3090 / P100: prefill 2.45× / 24.5×; decode 1.47× / 7.93×; capacity 3.33× / 6.67×.

Section 2.3 example. Split FP16 OPT-2.7B by decode compute: about 40% of parameters and KV cache on the 3090, 60% on the A100. The 3090 then has at most 18.6 GB for KV cache, while the A100 still has 44.1 GB unused.

The 10 GB figure on the previous slide is the FP16 7B case, not this 2.7B partition.

Source: Section 2.2, Table 1, and Section 2.3. Ratios are the paper's comparisons of these cells.

MLP and Attention Scale Differently



(a) MLP **(b) Attention**

Fig. 2: One Llama-70B decode layer; context length 1000

- The P100/A100 execution-time gap is much larger for MLP than for attention.
- Adding slow GPUs to dense operations can cost more communication than it saves in compute.
- Decode attention has a smaller performance gap and can use otherwise idle GPU memory.
- The preferred GPU partition can differ across modules within the same layer.

Use selected GPUs for dense work and a wider pool for decode attention.

Primary Workers and Attention Workers

HETIS / METHODOLOGY

Phase	Operation	Primary workers	Attention workers
Prefill	QKV / Output Projections, MLP	Execute	—
Prefill	Attention Core & KV Cache Creation	Execute	—
Decode	QKV / Output Projections, MLP	Execute	—
Decode	Attention Core	Process assigned heads	Process offloaded heads
Decode	KV Cache Storage	Store assigned heads	Store offloaded heads

$$\text{Attention core}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_h})V$$

Here, the attention core is parameter-free *after* Q, K and V are produced; the projections still require model weights.

Primary workers execute both prefill and decode, while attention workers only assist with decode attention and KV cache storage.

Source: Sections 2.4, 3.2 and 4. Attention core terminology clarified for this presentation.

Hetis System Architecture

- Parallelizer chooses Primary GPUs and their DP / PP / TP configuration.
- Profiler estimates GPU computation and communication cost.
- Dispatcher assigns heads to Primary and Attention workers, and selectively re-dispatches existing requests.
- Hauler manages head-level KV blocks and migrates the required cache shards.

Offline placement and profiling support online head allocation.

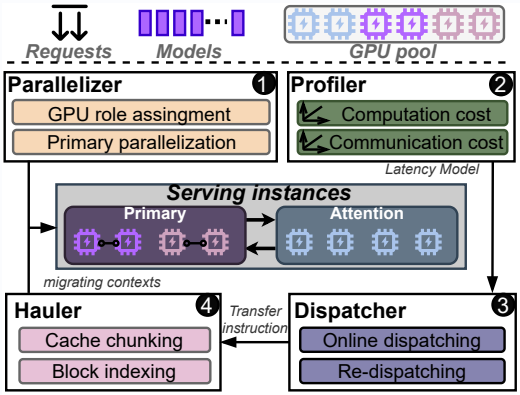


Fig. 3: Parallelizer, Profiler, Dispatcher and Hauler

Hierarchical Search for Primary Parallelism

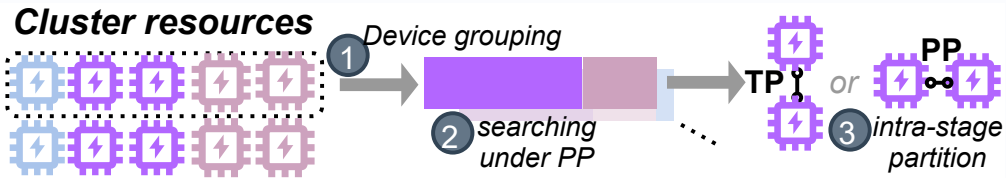


Fig. 4; Section 4.1: Primary worker configuration search

- **1. DP grouping:** divide each GPU type evenly across serving instances; filter insufficient KV capacity.
- **2. Coarse PP search:** treat each GPU type as one stage; assign layers to minimize the bottleneck compute cost.
- **3. GPU pruning:** try low-end GPUs first. Re-evaluate layer assignments on the remaining GPUs before the 5% test.
- **4. Fine-grained TP/PP search:** explore within each stage; score computation plus communication.

Excluded GPUs become a shared Attention-worker pool, rather than being left unused.

Primary Selection Objective and Pruning Rule

HETIS / METHODOLOGY

Objective (paper Eq. 1): choose placement and DP / PP / TP configuration σ.

$$\sigma^* = \arg\min_{\sigma} C(\sigma, M, \mathcal{R}), \qquad C = C_{\text{comp}} + C_{\text{comm}}$$

M: model; $\mathcal{R}$: workload distribution. **Pipeline bottleneck (explanatory):**

$$C_p = \max_s C_{\text{comp},s}$$

s: coarse PP stage. Ideal scaling; computation only, no communication. This restates the text, not a separately numbered paper equation.

GPU pruning (paper rule): re-evaluate layer assignments after removing κ.

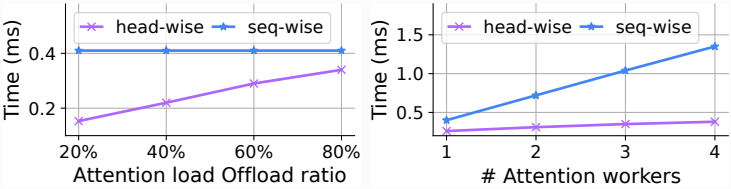
$$\frac{C_p(\sigma - \kappa, M, \mathcal{R})}{C_p(\sigma, M, \mathcal{R})} \leq 1 + \Delta, \qquad \Delta = 0.05.$$

Illustrative Example (assumed stage costs, arbitrary units): Before: $C_p = \max(100, 90, 20) = 100$. After removing P100 and reassigning layers: $C_p = \max(103, 98) = 103$. Ratio $1.03 \leq 1.05$: accept. Not measured results.

Reassigning layers changes the bottleneck cost and the pruning decision.

Source: Section 4.1, Eq. (1), Figure 4 and unnumbered pruning rule. Heuristic search; no global-optimality guarantee.

Why Split Attention by Head?



(a) Overhead under various loads **(b) Overhead across workers**

Fig. 5: Llama-70B, 100 Gbps; communication overhead

- Batch-wise splitting moves whole requests, which is coarse when context lengths differ.
- Sequence-wise splitting replicates query vectors across workers holding different token ranges.
- Head-wise splitting transfers only the assigned head vectors and their results.
- Reported communication speedups over sequence splitting: 2.68× at 20% offload; up to 3.55× with four workers.

Independent heads reduce communication and allow finer cache placement.

Head-wise Attention as Printed

HETIS / METHODOLOGY

$$\text{Attention}_j = \text{Concat}(\text{result}_{1,j}, \ldots, \text{result}_{n,j})$$

$$\text{result}_{i,j} = \text{softmax}\left(q_{h_i^j(t)} K_{h_i^j(t)}^{\top} / \sqrt{d}\right) V_{h_i^j(t)}$$

$h_i^j(t)$: query heads of request j placed on GPU i at time t. Each GPU finishes its own heads; outputs are concatenated in head order.

- This is Eq. (2). The earlier attention-core line is the same softmax, without the per-GPU split.
- No cross-GPU softmax reduction is required inside one head.

Head-wise splitting changes where each head runs, not the attention definition.

Source: Section 4.2, Eq. (2).

Dynamic Attention Placement Across Requests

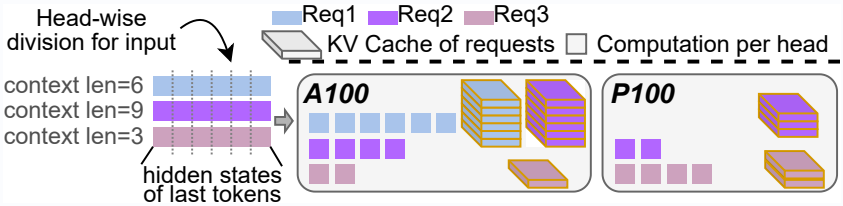


Fig. 6: Three requests, six heads each; context lengths 6, 9 and 3

- Each request can use a different head distribution across the same GPU pool.
- KV cache follows the assigned heads; longer contexts require more cache per head.
- Primary and Attention workers compute their assigned heads, then gather the head outputs.
- Re-dispatching changes selected assignments and migrates only the cache that must move.

Dynamic parallelism changes placement, not the number of model heads or the attention definition.

Head Integrity and Grouped-Query Attention

$$r = \frac{H_Q}{H_{KV}}, \qquad \sum_{i=1}^{N_k} x_i^j = H_Q, \qquad x_i^j / r \in \mathbb{Z}_{\geq 0}.$$

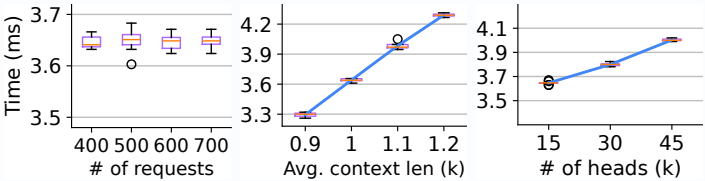
x_i^j: query heads of request j assigned to GPU i in pipeline stage k.

- MHA: $r = 1$; each query head has its own key/value head.
- GQA: each KV head is shared by a group of r query heads; Hetis preserves whole groups.
- Illustrative example: $H_Q = 8$, $H_{KV} = 2$, so $r = 4$; valid per-GPU allocations are 0, 4 or 8 query heads.
- Each assigned head uses its full context. Head-wise computation needs no cross-worker softmax reduction over sequence shards.

GQA reduces KV volume, but also makes the placement unit coarser than one query head.

Source: Sections 4.2, 5.2.1 and 6. Example is explanatory, not a measured model configuration.

Profiling Decode Attention Time



(a) Num. of requests **(b) context length** **(c) Num. of heads**

Fig. 7: OPT-30B attention micro-benchmarks

- With total head count and cache size fixed, changing the number of requests has little effect in this test.
- Attention time grows approximately linearly with cache size.
- At a fixed cache size, more query heads also increase execution time.
- These observations motivate a per-GPU affine model using head count and cache amount.

The model uses the work assigned to a GPU, not request count alone.

Computation and Communication Cost Models

HETIS / METHODOLOGY

$$\tau_i = a_i h_i + b_i g_i + c_i, \qquad \rho_i = \gamma_i d_i + \beta_i, \qquad d_i = (2 + 2/r) h_i.$$

- h_i: total query heads across requests; g_i: associated KV cache amount.
- a_i, b_i, c_i: fitted compute coefficients; γ_i, β_i: fitted transfer coefficients.
- Per decode step, remote heads need Q, new K/V and returned attention outputs: $2h_i$ query/output vectors plus $2h_i/r$ KV vectors.
- The expression for d_i counts vectors; head width and bytes per element must be accounted for when using physical byte units.

Attention-worker cost includes network transfer; retaining work on a Primary GPU avoids that extra transfer.

Source: Section 5.1, Eqs. (3)–(4). Vector-count interpretation makes the units explicit.

Online Dispatch for Newly Arriving Requests

HETIS / METHODOLOGY

Section 5.2.2 keeps dispatched placements fixed and assigns only new requests $\mathrm{J}(t)$.

Eq. (8): load update after solving the dispatch problem

$$h_i(t+1) = h_i(t) + \sum_{j=1}^{\mathrm{J}(t)} x_i^j(t),$$

$$g_i(t+1) = g_i(t) + \frac{2}{r} \sum_{j=1}^{\mathrm{J}(t)} x_i^j(t)\, l_j(t).$$

- $x_i^j(t)$: query heads of request j assigned to GPU i in stage k; N_k: workers in that stage.
- $h_i(t)$: existing query-head count; $g_i(t)$: existing cache amount; $l_j(t)$: request context length.
- r: query/KV-head ratio; $2/r$ accounts for K and V shared by each group.

Eq. (8) updates dispatch load; it does not specify all cache-growth or departure updates.

Source: Section 5.2.2, Eq. (8). Printed upper limit J(t) is retained; the prose defines J(t) as a set.

Printed Dispatch Constraints

$$\sum_{i=1}^{N_k} x_i^j(t) \in \{0, \mathrm{H}\}, \quad \forall j \in \mathbb{S}(t),\ \forall t. \tag{5}$$

$$\sum_{j \in \mathbb{S}(t)} x_i^j(t) l_j(t) \le \frac{r\mathrm{M}_i(t)}{2}, \quad \forall 1 \le i \le N_k,\ \forall t. \tag{6}$$

Eq. (7): only newly arriving requests

$$\min \max_i f_i(\vec{x}_i(t)), \tag{7a}$$

$$g_i(t) + \sum_{j=1}^{\mathrm{J}(t)} x_i^j(t) l_j(t) \le \frac{r\mathrm{M}_i(t)}{2}, \quad \forall i, t, \tag{7b}$$

$$\sum_{i=1}^{N_k} x_i^j(t) = \mathrm{H}, \quad \forall j \in \mathrm{J}(t). \tag{7c}$$

$\mathbb{S}(t)$: in-service requests; $\mathrm{J}(t)$: new requests; H: query heads; $\mathrm{M}_i(t)$: available KV capacity.

Keep printed (7b) unchanged: its cache scaling differs from Eq. (8).

Source: Section 5.2, Eqs. (5)–(7). Group integrality appears in the prose, not in printed (7).

Closed Forms of Attention Time

HETIS / METHODOLOGY

Primary worker: computation only (unnumbered, after Eq. (7))

$$f_i(\vec{x}_i(t)) = a_i\left(h_i(t) + \sum_{j=1}^{\mathrm{J}(t)} x_i^j(t)\right) + b_i\left(g_i(t) + \frac{2}{r}\sum_{j=1}^{\mathrm{J}(t)} l_j(t)x_i^j(t)\right) + c_i.$$

Attention worker: computation plus transfer (same paragraph)

$$f_i(\vec{x}_i(t)) = \left(a_i + \left(2 + \frac{2}{r}\right)\gamma_i\right)\left(h_i(t) + \sum_{j=1}^{\mathrm{J}(t)} x_i^j(t)\right) + b_i\left(g_i(t) + \frac{2}{r}\sum_{j=1}^{\mathrm{J}(t)} l_j(t)x_i^j(t)\right) + c_i + \beta_i.$$

a_i, b_i, c_i: compute coefficients; γ_i, β_i: transfer coefficients. The transfer term uses existing plus newly assigned heads; it sends Q, new K/V and outputs, not the entire historical cache each step.

Both expressions use the same $2/r$ cache increment as Eq. (8).

Source: Section 5.2.2, unnumbered formulas after Eq. (7). Time indices and printed sum limits restored.

Dispatch Formulation: What the Paper Specifies

HETIS / METHODOLOGY

Derived: epigraph form of the minimax objective only

$$\min_{x,z} z \quad \text{s.t.} \quad f_i(\vec{x}_i(t)) \leq z \quad \forall i.$$

- Retain the allocation and capacity constraints separately. This objective rewrite does not repair printed (7b).
- Section 5.2.1 requires $x_i^j(t)/r \in \mathbb{N}$. The paper does not define whether $0 \in \mathbb{N}$; Eq. (5) permits zero total allocation.
- Continuous variables give an LP relaxation; retaining group integrality gives a discrete problem. Section 5.2.2 claims LP reformulation but does not explain the integrality treatment.
- (7b) adds g_i to query-head-token counts; Eq. (8) and f_i multiply new counts by $2/r$. Cache units and normalization are not fully specified.

No claimed equivalence to a repaired capacity rule; no assumed rounding or solver algorithm.

Source: Section 5.2. Presenter analysis: J(t) is used as both a set and a sum limit; physical byte conversion is unspecified.

Re-dispatching to Balance Computation

HETIS / METHODOLOGY

Ideal attention time over existing requests $\mathbb{S}(t)$:

$$f^* = \min \max_i f_i(\vec{x}_i(t)) \quad \text{s.t.} \quad \sum_{i=1}^{N_k} \sum_{j \in \mathbb{S}(t)} x_i^j(t)\, l_j(t) \leq \sum_{i=1}^{N_k} \frac{r \cdot \mathrm{M}_i}{2}.$$

- If the deviation from the current attention time exceeds Θ (default 50%), pick the slowest GPU and its heaviest request.
- Re-dispatch that request with Eq. (7). Keep reusable KV shards; transfer only changed assignments.
- The displayed constraint is one cluster-wide sum. The paper does not define how "deviation" is normalized.

Growing contexts can create stragglers even if initial head placement was balanced.

Source: Section 5.3.1. Head-integrity constraints are not restated in this f^* display.

Re-dispatching to Relieve KV Cache Exhaustion

HETIS / METHODOLOGY

- A globally selected LIFO / LRU victim may hold no cache on the exhausted GPU.
- Restrict victim selection to requests occupying the overloaded device; the paper adapts LIFO to that device.
- Try redistribution when $\sum_i g_i(t) < \sum_i r \cdot \mathrm{M}_i/2$; otherwise the cluster has no aggregate free cache under the paper's units.
- Repeat subject to remaining cache capacity and the computation-imbalance threshold; reuse heads whose placement is unchanged.

Target the actual memory bottleneck and migrate only the necessary KV shards.

Source: Section 5.3.2. Aggregate free capacity alone does not guarantee a feasible discrete allocation.

Head-Level KV Management and Migration

HETIS / METHODOLOGY

CACHE LAYOUT

Split blocks along the head dimension

Extend vLLM's fixed-sized blocks; index by sequence ID, token position and head ID.

COMMUNICATION

NCCL point-to-point groups

Transfer head-specific inputs and results between Primary and Attention workers.

ATTENTION KERNEL

Reuse PagedAttention

Compute local heads independently; accelerate block indexing with CPU cores.

LIVE MIGRATION

Low-priority CUDA streams

Reduce interference with ongoing inference communication; bandwidth cost still exists.

Fine-grained placement requires matching cache indexing, kernels and transfer support.

Source: Section 6. Low-priority migration is not a guarantee of zero overhead.

Hetis Experimental Setup

CLUSTER

12 GPUs across four hosts

One host with 4 A100-80GB; two hosts with 2 RTX 3090 each; one host with 4 P100.

MODELS

MHA and GQA

Llama-13B and OPT-30B use MHA; Llama-70B uses GQA in the evaluation.

NETWORK

100 Gbps LAN; intra-host PCIe

A local heterogeneous cluster, rather than a cloud-rental cost comparison.

WORKLOADS

SG / HE / LB

ShareGPT: chatbot; HumanEval: code completion; LongBench: long-article summarization.

Compare Hetis with Splitwise and HexGen under the paper's specified deployments.

Source: Section 7.1. Table 1 lists 3090 with 24 GB and P100 with 12 GB.

Baselines and Evaluation Metrics

HETIS / EXPERIMENTAL RESULTS

- **Llama-70B deployment:** Hetis selects A100 / 3090 as Primary workers and P100 as Attention workers.
- **HexGen:** asymmetric parameter splitting with PP across GPU types and TP within stages.
- **Splitwise:** authors configure A100 TP=4 for prefill; decode uses two PP stages with TP=2 on 3090 and P100.
- **Figures 8–10:** normalized E2E latency in s/token vs. offered request rate. A later sharp rise indicates greater load tolerance.
- **Figures 12–13:** P95 latency normalized to Hetis, which is 1.0. These are ratios, unlike the s/token axes above.

Keep deployment assumptions and normalization conventions explicit when interpreting speedups.

Source: Section 7.2. No fixed SLO goodput threshold or exact E2E normalization formula is specified.

Llama-13B: Latency Under Increasing Load

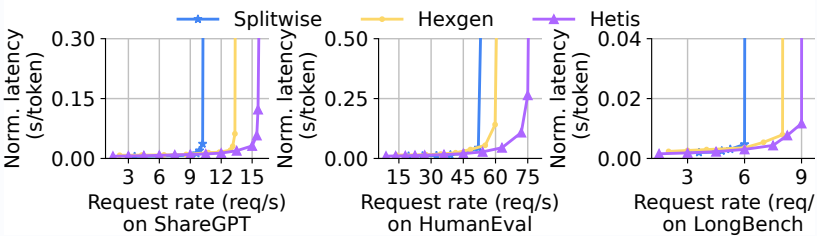


Fig. 8: SG / HE / LB; normalized E2E latency in s/token

- Compare methods within each dataset at the same offered request rate.
- Hetis delays the sharp latency increase across all three workload curves.
- The saturation points differ by dataset; each panel has its own axis scale.
- The advantage is most visible as load increases and compute or cache capacity becomes limiting.

The figure supports higher sustainable load, not a universal per-request latency ratio.

OPT-30B: Latency Under Increasing Load

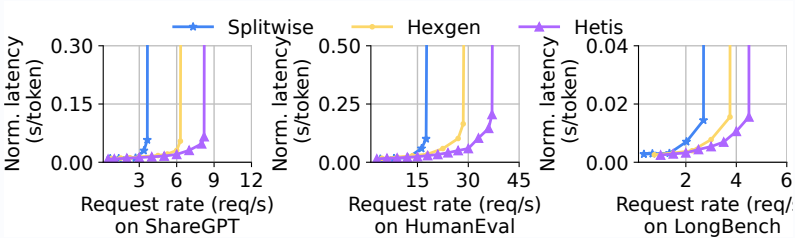


Fig. 9: OPT-30B (MHA), three workload families

- Hetis also delays the rapid latency increase for the larger MHA model.
- With larger model weights, placement determines how much capacity remains for KV cache.
- Head allocation can use memory that static compute-based partitioning leaves underutilized.
- Compare curves at matched load; do not compare raw heights across different dataset panels.

The benefit persists for OPT-30B in the evaluated heterogeneous cluster.

Llama-70B: GQA End-to-End Results

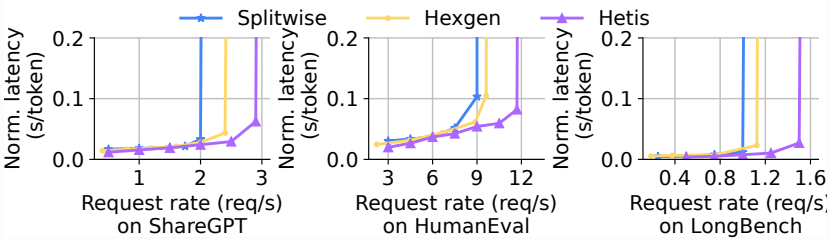


Fig. 10: Llama-70B; Section 7.2 summarizes Figures 8–10

- Hetis retains the advantage with a GQA model, whose KV cache per query head is smaller.
- A100 / 3090 handle dense work; P100 contributes decode attention and cache capacity.
- Across the end-to-end experiments, reported throughput gains are up to 2.25× vs. Splitwise and 1.33× vs. HexGen.
- These maxima summarize the experiments; they are not guaranteed for every panel or workload.

Module placement and dynamic attention sharing remain useful with GQA.

Maximum Usable KV Cache Space

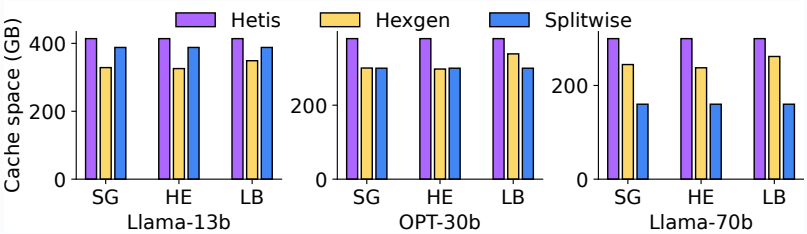


Fig. 11: Maximum available KV cache during inference

- Hetis reports up to 1.87× more usable KV cache capacity than the baselines.
- It avoids storing full model copies on attention-only workers.
- Dynamic allocation reduces the idle cache caused by fixed compute-based partitions.
- The gain is usable cache capacity, not additional physical GPU memory or a guaranteed equal throughput gain.

More usable cache allows the system to host more concurrent request state.

P95 TTFT and TPOT on Llama-70B

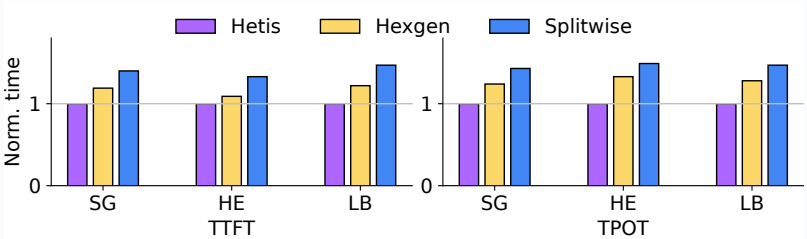


Fig. 12: SG 1.5 req/s; HE 6 req/s; LB 0.8 req/s

- All baselines are below saturation at these selected request rates.
- P95 TTFT ratio: up to 1.22× vs. HexGen and 1.47× vs. Splitwise.
- P95 TPOT ratio: up to 1.39× over the baselines.
- Primary selection helps prefill; avoiding slow dense execution and sharing attention help decode.

The tail-latency comparison tests unsaturated operation, beyond maximum load capacity.

MLP and Attention Contributions to Decode Latency

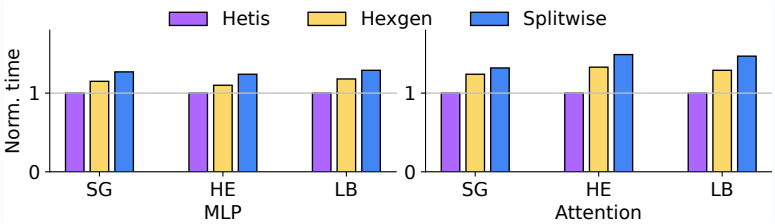


Fig. 13: Llama-70B; the same request rates as Fig. 12

- MLP P95 latency improves by up to 1.29×; attention by up to 1.49×.
- Dense and attention modules use different placement decisions.
- For this experiment, module latency is the maximum stage time multiplied by the number of PP stages.
- The 1.49× is a module-level result here; it should not be relabeled as a universal E2E speedup.

Selective dense parallelism and dynamic attention sharing contribute separately.

Dynamic Head and Cache Allocation

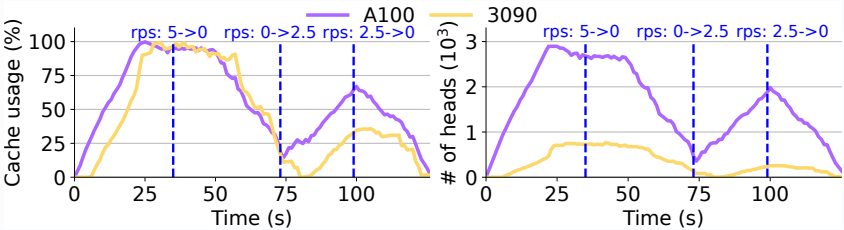
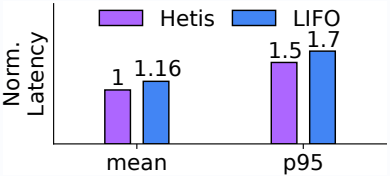


Fig. 14: Llama-13B; one A100 Primary + two 3090 Attention GPUs

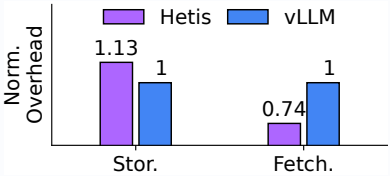
- ShareGPT arrival rate changes over time: $5 \rightarrow 0 \rightarrow 2.5 \rightarrow 0$ req/s.
- A100 processes more heads, while both GPU types can reach high cache utilization.
- 3090 participation starts later under light load because remote execution adds communication cost.
- The head counts and cache usage change with both arrivals and completion of ongoing requests.

Hetis activates remote attention when its compute and memory benefits justify communication.

Re-dispatching Benefits and Cache Operation Costs



(a) Benefit of re-dispatching



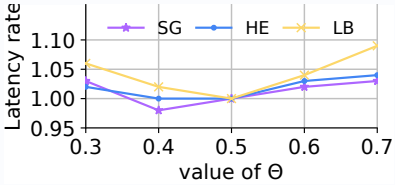
(b) Head management overhead

Fig. 15: Re-dispatching vs. LIFO; head management vs. vLLM

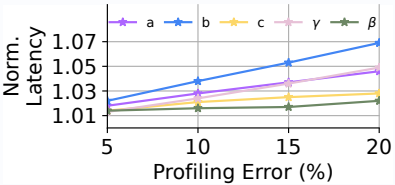
- ShareGPT at 5 req/s: re-dispatching improves output latency relative to LIFO.
- P95 ratio is about 1.14×; mean ratio is inconsistent: text 1.06×, plotted bars 1.16×.
- Head-wise cache storage-operation overhead rises by 13%; fetching time falls by 26%.
- The 13% is not 13% more KV bytes: the paper states equal KV memory consumption.

Re-dispatching has benefits and implementation costs; preserve the metric and comparison.

Sensitivity to Threshold and Profiling Error



(a) Sensitivity to value of Θ



(b) Sensitivity to profiling error

Fig. 16: Re-dispatch threshold and fitted-model perturbations

- Default $\Theta = 0.5$ balances migration frequency against computation imbalance.
- Smaller thresholds can migrate too often; larger thresholds can leave stragglers uncorrected.
- Perturbing fitted coefficients by up to $\pm 20\%$ raises per-token latency by at most 6.9% in the reported tests.
- This evaluates model-coefficient error, not all network failures or workload shifts.

The tested sensitivity is moderate; it is not a general robustness guarantee.

Planning and Profiling Overhead

Component	Setting	Reported result
Primary placement search	Local heterogeneous cluster	4 seconds
Larger placement search	Simulation: 5 types $\times$ 32 GPUs	15 seconds
Profiling samples	8 head counts $\times$ 8 cache amounts	Typically $\leq$ 100 ms per configuration
Transfer model accuracy	Profiled settings	92.4–96.1%

- Placement search runs before initial deployment; the 160-GPU result is a simulation.
- Computation-model accuracy is reported as up to 93.8%, without a precise accuracy formula.
- These timings do not establish the latency distribution of the online dispatch solver.

Separate one-time planning cost, per-sample profiling cost and online scheduling overhead.

Source: Section 7.4.

Hetis: Scope and Open Questions

HETIS / DISCUSSION

- **Granularity:** GQA groups restrict how finely heads and memory can be divided; integer handling is underspecified.
- **Network:** repeated per-layer input/result transfers and occasional KV migration may behave differently on slower or congested links.
- **Evidence:** end-to-end tests use a local 12-GPU cluster; the larger-scale result measures simulated placement search.
- **Reproduction:** memory-unit conventions, E2E normalization and online solver latency need more detail.

Research question: how should head placement change when memory capacity grows faster than memory bandwidth?

Presenter analysis based on Sections 4–7; the research question is not evaluated in Hetis.

Hetis: Main Contributions and Findings

- **Primary parallelism:** select GPUs and DP / PP / TP configurations for dense work, including execution through both inference phases.
- **Dynamic attention:** assign whole head groups per request across Primary and Attention workers using compute, network and cache models.
- **Runtime support:** selectively re-dispatch growing requests and migrate only necessary head-level KV shards.
- **Reported results:** throughput up to 2.25× vs. Splitwise and 1.33× vs. HexGen; usable KV capacity up to 1.87×.

Different modules need different GPU partitions, and attention placement should adapt online.

Source: Sections 3–7 and 9. Ratios are maxima in the evaluated settings.

ThunderServe vs. Hetis: Design Choices

CROSS-PAPER COMPARISON

Dimension	ThunderServe	Hetis
Granularity	Prefill / decode phases	Dense modules / attention heads
Workers	Phase-specific model replicas	Primary and Attention workers
Search	Tabu search + lower-level selection	Primary search + head dispatch
Runtime changes	Phase roles and routing	Head allocation / redispatch
Data movement	Inter-replica KV transfer	Per-layer Q/K/V/results; KV migration
Metrics	SLO attainment, throughput	E2E s/token, P95, usable KV, request rate

ThunderServe and Hetis are not compared head-to-head in these papers.

Sources: ThunderServe Sections 3–5; Hetis Sections 4–7.

Implications for GPU + HBM / HBF Co-design

CROSS-PAPER / RESEARCH DIRECTIONS

QUESTION 1

Memory-aware placement

How do HBM bandwidth and capacity alter phase or head assignment?

QUESTION 2

Tiered KV caches

Could a slower, larger tier improve capacity without hurting TPOT?

QUESTION 3

Serving simulation

What hardware-model extensions are needed in LLMServingSim?

These are follow-up research hypotheses, not evaluated HBF results in either paper.

Discussion focus: scheduling policy, cost-model fidelity, and memory hierarchy.

Questions & Discussion

GPU heterogeneity / KV cache / HBM–HBF

ThunderServe (phase level) vs. Hetis (head level)